

Annexe de compréhension du TIPE

Optimisation des tournées de collecte des déchets urbains

Document de travail personnel — tout y est expliqué depuis zéro,
puis ramené au niveau exigé par le jury

Brahim AMSEGT — candidat n°13530 — CPGE MP — session 2026

Comment lire ce document. Il n'est pas ta MCOT et ne sera pas montré au jury. C'est ton outil pour comprendre *réellement* ce que tu présentes. Chaque notion part de l'intuition la plus simple (aucune connaissance préalable en théorie des graphes supposée), puis monte jusqu'à la formulation rigoureuse attendue en MP. Les encadrés **rouges** signalent les questions que le jury posera et ce qu'il faut répondre. Les mentions « à vérifier » indiquent une affirmation que tu dois confirmer dans la source originale avant de la défendre.

Table des matières

1	La théorie des graphes en partant de rien	2
1.1	Un graphe, qu'est-ce que c'est ?	2
1.2	Le vocabulaire minimal	2
1.3	Graphe complet, et pourquoi	2
1.4	Comment un ordinateur « voit » un graphe : la matrice d'adjacence	3
1.5	Chemins, cycles, et les deux grands « cousins » : Euler et Hamilton	4
1.6	Du camion-poubelle au cycle hamiltonien (et l'honnêteté sur le modèle)	5
2	Le problème du voyageur de commerce (TSP)	5
2.1	L'énoncé, en une phrase	5
2.2	Combien y a-t-il de tours possibles ? La démonstration de $(n-1)!/2$	5
2.3	L'explosion combinatoire : pourquoi on ne peut pas tout essayer	6
2.4	NP-complétude : ce que ça veut dire vraiment	6
3	Algorithme 1 — Le plus proche voisin (PPV)	7
3.1	L'idée, en une phrase	7
3.2	Déroulé chiffré sur le graphe fil rouge	7
3.3	Le code Python	8
3.4	Pourquoi la complexité est $O(n^2)$	8
3.5	La faiblesse : myopie et arête de retour	9
4	Algorithme 2 — L'amélioration locale 2-opt	9
4.1	L'idée : décroiser le tour	9
4.2	Le mouvement 2-opt et le calcul du gain Δ	9
4.3	Le code Python	10
4.4	Preuve que l'algorithme se termine (à savoir par cœur)	11
4.5	Complexité, et une honnêteté importante	11
4.6	Optimum local $\neq$ optimum global (le point crucial)	12
5	Algorithme 3 — Held–Karp (résolution exacte)	12
5.1	Le gâchis de la force brute	12
5.2	La bonne idée : raisonner sur des sous-ensembles	12
5.3	L'équation de récurrence (Bellman)	13
5.4	Comment l'ordinateur représente un sous-ensemble : le « bitmask »	13
5.5	Déroulé entièrement chiffré sur 4 points	13

5.6	Le code Python	14
5.7	Complexité : pourquoi le mur à $n \approx 20$	14
6	Synthèse et lien avec le thème « Cycles, boucles »	15
6.1	Le tableau comparatif (à connaître par cœur)	15
6.2	Le triple ancrage au thème	15
7	Glossaire express	15
8	Antiséche pour l'oral : 8 questions, 8 réponses	16

1 La théorie des graphes en partant de rien

1.1 Un graphe, qu'est-ce que c'est ?

Intuition

Regarde un plan de métro. Il y a des *stations* (des points) et des *lignes* qui relient certaines stations entre elles. Tu te fiches de la forme exacte des tunnels : ce qui compte, c'est *quelles* stations sont reliées et *combien* ça coûte (en temps) de passer de l'une à l'autre. Un **graphe**, c'est exactement ça : des points, et des liens entre ces points. Rien de plus.

Dans notre TIPE, les *points* sont les endroits où le camion s'arrête pour ramasser les ordures (bacs, conteneurs, regroupements de poubelles). Les *liens* sont les trajets routiers possibles entre deux arrêts. C'est tout le saut : une réalité urbaine compliquée (rues, virages, feux) devient une structure mathématique propre sur laquelle on peut calculer.

1.2 Le vocabulaire minimal

Définition — graphe pondéré

Un **graphe pondéré** est un triplet $G = (V, E, w)$ où :

- V (de l'anglais *vertices*) est l'ensemble des **sommets** — nos points de collecte ;
- E (de *edges*) est l'ensemble des **arêtes** — les trajets reliant deux sommets ;
- $w : E \rightarrow \mathbb{R}_+$ (de *weight*) est la fonction de **poids** : $w(u, v)$ est le coût pour aller de u à v .

On note $n = |V|$ le **nombre de sommets** (aussi appelé *ordre* du graphe).

C'est *exactement* la notation de ta MCOT : $G = (V, E, w)$. Tu dois pouvoir réciter ce triplet et dire ce que chaque lettre représente *dans ton problème concret*.

À retenir

Dans mon TIPE : V = les 30 à 40 points de collecte du quartier ; E = les trajets entre points ; $w(u, v)$ = le *temps* de trajet (pas la distance brute — voir §1.6).

1.3 Graphe complet, et pourquoi

On dit qu'un graphe est **complet** quand *toute* paire de sommets est reliée par une arête. Le graphe complet à n sommets se note K_n .

Intuition

Pourquoi supposer le graphe complet, alors que dans une vraie ville on ne peut pas aller en ligne droite d'un point à n'importe quel autre ? Parce qu'on raisonne sur les *trajets*, pas sur

les rues : entre deux points de collecte, il existe *toujours* un chemin routier, et son temps de parcours est le poids de l'arête. On « résume » tout l'itinéraire réel entre u et v par un seul nombre $w(u, v)$.

Graphe complet pondéré K_6 : 6 points de collecte
(arête = trajet possible, étiquette = coût)

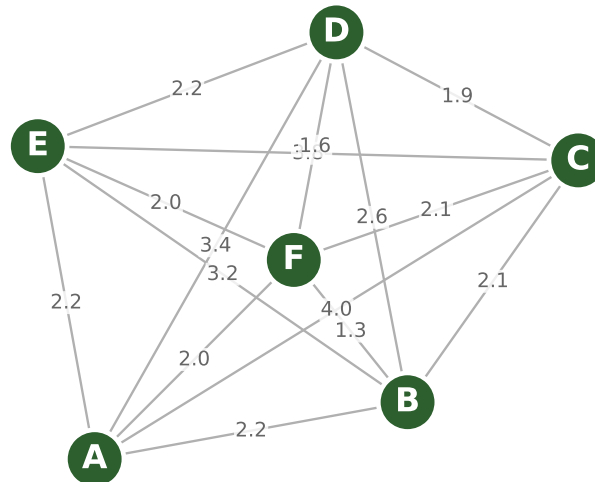


Figure 1: Le petit graphe complet K_6 qui servira d'exemple fil rouge dans tout ce document : six points de collecte, toutes les paires reliées, chaque arête étiquetée par son coût.

1.4 Comment un ordinateur « voit » un graphe : la matrice d'adjacence

L'ordinateur ne voit pas le dessin. Il manipule un tableau de nombres. La façon la plus simple de lui décrire un graphe complet pondéré est la **matrice d'adjacence** (ici, plutôt matrice des *distances/coûts*) : un tableau carré D de taille $n \times n$ où la case $D[i][j]$ contient le coût $w(i, j)$ pour aller du sommet i au sommet j .

Intuition

C'est la même idée qu'un *tableau des kilométrages* au dos d'une carte routière : on lit la ligne « Meknès » et la colonne « Fès », et la case donne la distance. La matrice d'adjacence, c'est ce tableau, pour tous nos points de collecte.

Propriétés utiles à connaître pour le jury :

- la diagonale est nulle : $D[i][i] = 0$ (aller d'un point à lui-même ne coûte rien) ;
- si le coût ne dépend pas du sens ($w(u, v) = w(v, u)$), la matrice est **symétrique** ; on parle alors de graphe *non orienté*. C'est l'hypothèse qu'on fait (à défendre : §1.6).

Le code Python ci-dessous construit D à partir des coordonnées des points (ici, distance « à vol d'oiseau », mais on remplacera par des temps de trajet réels) :

```
1 import math
2
3 def build_distance_matrix(points):
4     """points : dict {indice : (x, y)}. Retourne la matrice n x n des couts."""
5     n = len(points)
6     D = [[0.0] * n for _ in range(n)]
```

```

7   for i in range(n):
8       for j in range(n):
9           (xi, yi), (xj, yj) = points[i], points[j]
10          D[i][j] = math.hypot(xi - xj, yi - yj) # = sqrt((xi-xj)^2+(yi-yj)^2)
11  return D

```

Pour notre graphe fil rouge (figure 1), cette matrice vaut concrètement (coûts arrondis au centième) :

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
<i>A</i>	0	2,24	4,00	3,45	2,24	1,98
<i>B</i>	2,24	0	2,08	2,65	3,16	1,28
<i>C</i>	4,00	2,08	0	1,92	3,80	2,12
<i>D</i>	3,45	2,65	1,92	0	2,25	1,63
<i>E</i>	2,24	3,16	3,80	2,25	0	1,97
<i>F</i>	1,98	1,28	2,12	1,63	1,97	0

On lit par exemple $w(A, C) = 4,00$; la diagonale est nulle ; la matrice est symétrique. **Tous les nombres chiffrés de ce document se retrouvent dans cette table** : tu peux donc vérifier chaque calcul toi-même.

1.5 Chemins, cycles, et les deux grands « cousins » : Euler et Hamilton

Définition — chemin, cycle

Un **chemin** est une suite de sommets reliés de proche en proche par des arêtes. Un **cycle** est un chemin qui revient à son point de départ. (Le camion part du dépôt, fait sa tournée, revient au dépôt : c'est un cycle.)

Il faut distinguer deux types de parcours, car ta MCOT les évoque tous les deux et le jury testera si tu ne les confonds pas :

	Cycle eulérien	Cycle hamiltonien
On passe par chaque...	arête exactement une fois	sommet exactement une fois
Question type	« tracer sans lever le crayon »	« tournée visitant tous les points »
Existence	facile : critère d'Euler (1736)	difficile : pas de critère simple
Lien TIPE	ponts de Königsberg (origine historique)	c'est NOTRE problème

Intuition

Euler (les ponts de Königsberg, 1736) cherche à parcourir chaque *pont* une fois : c'est une histoire d'*arêtes*. **Hamilton** (jeu icosien, 1857 — à vérifier) cherche à visiter chaque *sommet* une fois : c'est une histoire de *points*. La collecte des ordures, si on la modélise par des points à visiter, est un problème *hamiltonien*.

À retenir

Euler donne un critère limpide : un graphe connexe admet un cycle eulérien **si et seulement si tous ses sommets sont de degré pair** (le degré = nombre d'arêtes qui touchent le sommet). Pour Hamilton, *aucun* critère aussi simple n'existe — et c'est précisément ce qui

rend notre problème difficile. Cette opposition « facile / difficile » est le cœur de la justification de tout le TIPE.

1.6 Du camion-poubelle au cycle hamiltonien (et l'honnêteté sur le modèle)

Voici le pont entre la réalité et les maths, étape par étape :

1. chaque point de collecte $\rightarrow$ un sommet de V ;
2. chaque trajet possible entre deux points $\rightarrow$ une arête de E ;
3. le temps de parcours d'un trajet $\rightarrow$ le poids $w(u, v)$;
4. « faire une tournée qui ramasse partout en revenant au dépôt et minimise le temps » $\rightarrow$ **trouver le cycle hamiltonien de poids minimal**.

Pour le poids, on choisit le **temps** plutôt que la distance, car deux trajets de même longueur peuvent prendre des temps très différents (montée, feux, type de voie). Modèle :

$$t(u, v) = \frac{d(u, v)}{\bar{v}(\text{type de voie})} + \text{pénalité moyenne de trafic.}$$

On simplifie en supposant le trafic négligeable pendant la collecte (5h–7h à Meknès), ce qui rend $t(u, v)$ **déterministe** (un seul nombre, pas une variable aléatoire).

Question piège du jury

« La collecte des ordures, c'est parcourir des *rues*, pas visiter des *points*. Pourquoi un TSP et pas un CARP ? »

C'est LA question à préparer. Trois problèmes voisins existent :

- **TSP** (*Travelling Salesman*) : visiter des *sommets* — c'est mon modèle ;
- **VRP** (*Vehicle Routing*) : TSP + plusieurs véhicules + *capacité* limitée ;
- **CARP** (*Capacitated Arc Routing*) : on doit parcourir des *arêtes* (toutes les rues), pas seulement visiter des points.

Réponse à donner : « Le vrai problème de collecte est un CARP. Je l'ai *volontairement* simplifié en TSP car (1) avec des points de collecte denses, regrouper chaque rue en un point représentatif est une approximation raisonnable ; (2) le TSP est au programme MP en termes d'outils (graphes, complexité, programmation dynamique) alors que le CARP exige des outils hors-programme ; (3) je l'assume comme une *hypothèse de modélisation explicite*, pas comme une méconnaissance. » Ne JAMAIS prétendre que c'est exactement la réalité.

2 Le problème du voyageur de commerce (TSP)

2.1 L'énoncé, en une phrase

Définition — TSP (problème du voyageur de commerce)

Étant donné un graphe complet pondéré K_n , trouver le **cycle hamiltonien de poids total minimal** : l'ordre de visite des n sommets (chacun une fois, retour au départ) qui minimise la somme des poids des arêtes empruntées.

2.2 Combien y a-t-il de tours possibles ? La démonstration de $(n-1)!/2$

C'est un résultat que tu *dois* savoir redémontrer au tableau en 30 secondes.

Intuition

On a n points à mettre dans un certain ordre. Le nombre d'ordres (permutations) de n objets est $n! = n \times (n-1) \times \dots \times 1$. Mais sur un *cycle*, plusieurs ordres décrivent la même tournée.

Démonstration. Il y a $n!$ façons d'ordonner n sommets. Mais :

- un cycle n'a pas de « début » : on peut le faire commencer par n'importe lequel de ses n sommets sans changer la tournée $\Rightarrow$ on divise par n ;
- un cycle peut être parcouru dans les *deux sens* ($A \rightarrow B \rightarrow C$ ou $A \rightarrow C \rightarrow B$) pour le même coût (graphe non orienté) $\Rightarrow$ on divise par 2.

D'où le nombre de cycles hamiltoniens distincts :

$$\frac{n!}{2n} = \frac{(n-1)!}{2}. \quad \blacksquare$$

2.3 L'explosion combinatoire : pourquoi on ne peut pas tout essayer

Mettons des chiffres dessus (ta MCOT en cite, sache les justifier) :

$$n = 10 : \frac{9!}{2} \approx 1,8 \times 10^5; \quad n = 15 : \frac{14!}{2} \approx 4,4 \times 10^{10}; \quad n = 20 : \frac{19!}{2} \approx 6,1 \times 10^{16}.$$

Dès $n = 15$, on dépasse les 43 milliards de tours (le chiffre de ta MCOT). À 10^9 tours évalués par seconde, $n = 20$ demanderait déjà près de deux ans de calcul. **Énumérer tous les tours est donc impossible** au-delà d'une douzaine de sommets : c'est ce qu'on appelle l'explosion combinatoire (figure 2).

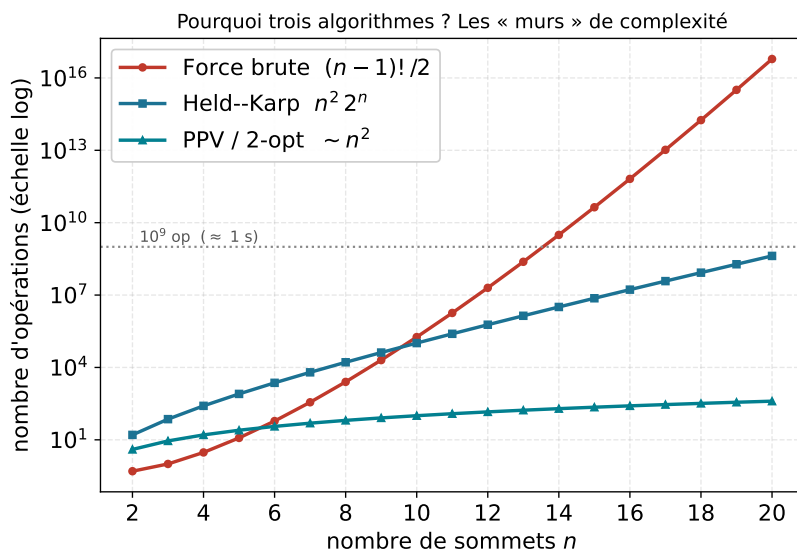


Figure 2: Les trois régimes de complexité (échelle logarithmique). La force brute (rouge) franchit le mur des « calculs raisonnables » vers $n \approx 15$; Held-Karp (bleu) le repousse à $n \approx 20$; les heuristiques polynomiales (vert) restent praticables très loin.

2.4 NP-complétude : ce que ça veut dire vraiment

Intuition

Un problème est *facile* si on connaît un algorithme dont le temps croît comme une *puissance* de n (par ex. n^2) : on dit **polynomial**. Il est *difficile* si tous les algorithmes connus croissent comme une *exponentielle* (2^n , $n!$...), qui explose bien plus vite. La classe **NP-complet** re-

groupe les problèmes les plus durs parmi ceux dont on peut *vérifier* rapidement une solution proposée.

À retenir

En 1972, Richard KARP place le problème du cycle hamiltonien dans sa liste de 21 problèmes **NP-complets**. La version *décision* du TSP (« existe-t-il un tour de coût $\leq k$? ») est NP-complète ; la version *optimisation* (trouver le minimum) est NP-difficile. Conséquence pratique : **on ne connaît aucun algorithme polynomial qui résolve le TSP exactement**, et on pense (sans preuve : c'est la question $P \stackrel{?}{=} NP$, non résolue) qu'il n'en existe pas.

Question piège du jury

« **NP-complet, ça veut dire impossible à résoudre ?** » Non. Ça veut dire qu'on ne sait pas le résoudre *exactement* en temps polynomial. On a deux échappatoires, et mon TIPE explore les deux : soit on résout *exactement* mais lentement (Held–Karp, limité aux petites instances) ; soit on résout *vite* mais sans garantie d'optimalité (heuristiques : plus proche voisin, 2-opt).

3 Algorithme 1 — Le plus proche voisin (PPV)

3.1 L'idée, en une phrase

Intuition

« À chaque arrêt, je vais au point le plus proche que je n'ai pas encore visité. » C'est la stratégie du gourmand pressé : on prend toujours le meilleur coup *immédiat*, sans réfléchir aux conséquences. On dit que l'algorithme est **glouton** (*greedy*).

Algorithme du plus proche voisin (heuristique gloutonne)

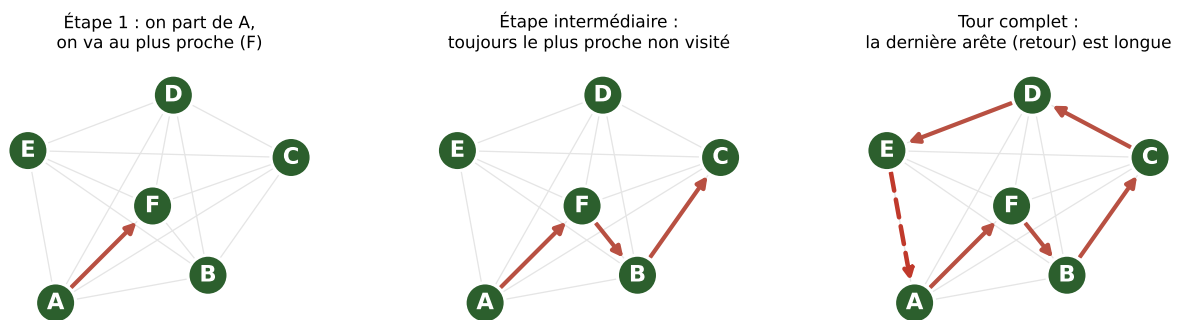


Figure 3: Le plus proche voisin sur le graphe fil rouge, depuis A. On enchaîne les sauts les plus courts. . . jusqu'à ce qu'il ne reste que des points lointains : la dernière arête de retour (en pointillé rouge) est coûteuse. C'est la faiblesse structurelle de la méthode.

3.2 Déroulé chiffré sur le graphe fil rouge

Partons de A. À chaque étape on regarde la ligne de la matrice D du sommet courant et on choisit le plus petit coût vers un sommet non visité. Sur le graphe de la figure 1, l'exécution réelle donne

le tour

$$A \rightarrow F \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A,$$

de longueur totale $\approx 11,75$. (Sur ce petit exemple précis, ce tour se trouve être *déjà optimal* — on le vérifiera avec Held-Karp. C'est une chance, pas une garantie !)

Déroulé entièrement chiffré (à suivre une fois, calculatrice en main). À chaque ligne, on lit la ligne de la matrice D du sommet où l'on se trouve, on barre les sommets déjà visités, et on garde le plus petit coût restant.

On est en	Coûts vers les sommets <i>non encore visités</i>					On choisit
A	F= 1,98	B= 2,24	E= 2,24	D= 3,45	C= 4,00	F (1,98)
F	B= 1,28	D= 1,63	E= 1,97	C= 2,12		B (1,28)
B	C= 2,08	D= 2,65	E= 3,16			C (2,08)
C	D= 1,92	E= 3,80				D (1,92)
D	E= 2,25	(seul restant)				E (2,25)
E	retour obligatoire au départ A					A (2,24)

Longueur totale, en additionnant la colonne de droite :

$$1,98 + 1,28 + 2,08 + 1,92 + 2,25 + 2,24 = 11,75.$$

Remarque le mécanisme de la *myopie* : au début on prend des arêtes très courtes (1,98 puis 1,28), mais l'arête de *retour* $E \rightarrow A$ (2,24) n'a pas pu être choisie : elle est *subie*. Sur une instance moins favorable, ce retour forcé peut être catastrophique.

3.3 Le code Python

```
1 import math
2
3 def nearest_neighbor(D, start=0):
4     """Construit un tour par la règle du plus proche voisin, depuis 'start'."""
5     n = len(D)
6     tour = [start]
7     visited = [False] * n
8     visited[start] = True
9     current = start
10    for _ in range(n - 1): # il reste n-1 sommets à placer
11        best_dist, best_j = math.inf, -1
12        for j in range(n): # on cherche le plus proche non visité
13            if not visited[j] and D[current][j] < best_dist:
14                best_dist, best_j = D[current][j], j
15        tour.append(best_j)
16        visited[best_j] = True
17        current = best_j
18    return tour # le retour au départ est implicite
```

3.4 Pourquoi la complexité est $O(n^2)$

Intuition

On répète $n - 1$ fois une opération (« chercher le plus proche ») qui passe en revue les n sommets. « n fois quelque chose qui coûte n », ça fait de l'ordre de n^2 .

Comptage précis. La boucle externe (ligne 9) s'exécute $n - 1$ fois. À chaque tour, la boucle interne (ligne 11) parcourt les n sommets. Le nombre total d'opérations élémentaires est donc de l'ordre de $(n - 1) \times n$, soit $\Theta(n^2)$.

3.5 La faiblesse : myopie et arête de retour

À retenir

Le PPV est **myope** : il optimise chaque pas isolément, sans anticiper. Les premiers sauts sont courts (on a l'embarras du choix), mais à la fin il ne reste que des sommets éloignés, et surtout il faut *revenir au départ* par une arête souvent très longue (le pointillé rouge de la figure 3). **Aucune garantie d'optimalité.**

Question piège du jury

« **Votre résultat dépend-il du point de départ ?** » Oui, fortement : partir de A ou de C peut donner des tours très différents. **Parade que j'implémente** : lancer le PPV depuis *chacun* des n sommets et garder le meilleur tour. Coût : n exécutions à $O(n^2)$, soit $O(n^3)$ au total — toujours polynomial, donc acceptable.

Question piège du jury

« **Le PPV a-t-il une garantie de qualité ?** » Pour un TSP *métrique* (les poids vérifient l'inégalité triangulaire), Rosenkrantz, Stearns et Lewis (1977) ont montré que le PPV ne dépasse jamais l'optimum d'un facteur $\frac{1}{2}(\lceil \log_2 n \rceil + 1)$ (*constante exacte à vérifier dans l'article original*). Donc le tour peut être de l'ordre de $\log n$ fois trop long dans le pire cas : c'est une garantie faible, qui justifie de *l'améliorer* ensuite par 2-opt.

4 Algorithme 2 — L'amélioration locale 2-opt

4.1 L'idée : décroiser le tour

Intuition

Prends une tournée existante (par ex. celle du PPV) tracée sur une carte. Si deux segments du tour *se croisent*, c'est qu'on fait un détour inutile. En « décroisant » ces deux segments, on raccourcit toujours le tour. 2-opt cherche systématiquement ces croisements et les défait, l'un après l'autre, jusqu'à ce qu'il n'y en ait plus.

4.2 Le mouvement 2-opt et le calcul du gain Δ

Concrètement, un **mouvement 2-opt** retire deux arêtes du tour et les remplace par deux autres. Si le tour passe $\dots a \rightarrow b \dots c \rightarrow d \dots$, on retire (a, b) et (c, d) , et on ajoute (a, c) et (b, d) ; le segment entre b et c est alors parcouru à *l'envers* (figure 5).

Le **gain** (variation de longueur) se calcule *sans recalculer tout le tour*, juste avec les quatre arêtes concernées :

$$\Delta = \underbrace{(w(a, c) + w(b, d))}_{\text{nouvelles arêtes}} - \underbrace{(w(a, b) + w(c, d))}_{\text{anciennes arêtes}}.$$

Si $\Delta < 0$, le mouvement raccourcit le tour : on l'applique. Sinon on le rejette.

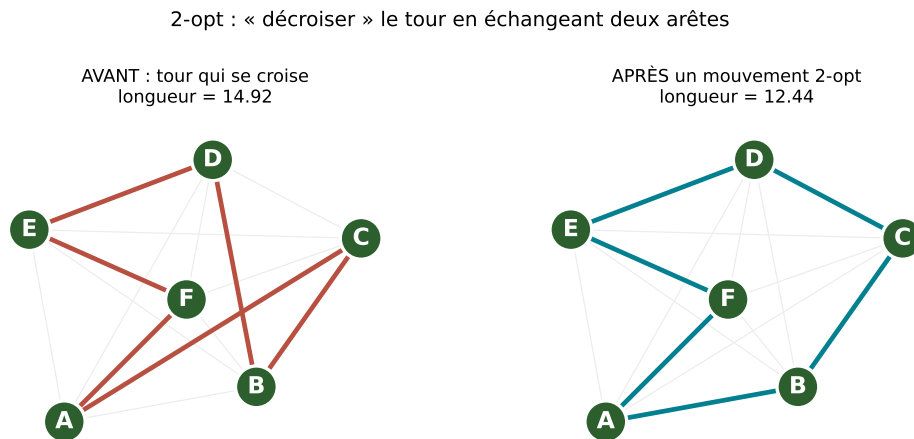


Figure 4: Un tour qui se croise (gauche, coût 14,92) et le même tour après *un* mouvement 2-opt (droite, coût 12,44). Le croisement a disparu et le tour est plus court.

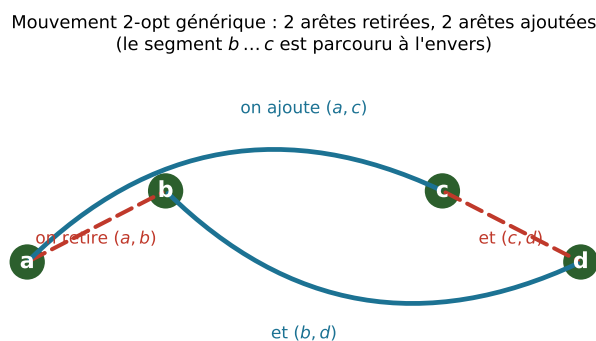


Figure 5: Le mouvement générique : deux arêtes retirées (rouge pointillé), deux ajoutées (bleu).

À retenir

Le coup de génie pratique : on n'a pas besoin de recalculer la longueur entière du tour à chaque essai (ce serait $O(n)$ par essai). Il suffit de comparer *quatre* arêtes : Δ se calcule en temps *constant* $O(1)$.

Le calcul du gain, en vrai, sur un tour précis. Partons du tour croisé $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow A$, de longueur 14,92 (c'est le tour de gauche de la figure 4). On teste le mouvement qui retire les arêtes (A, C) et (B, D) et ajoute (A, B) et (C, D) :

$$\begin{aligned}
 \text{on retire : } & w(A, C) + w(B, D) = 4,00 + 2,65 = 6,65, \\
 \text{on ajoute : } & w(A, B) + w(C, D) = 2,24 + 1,92 = 4,16, \\
 \Delta = \text{ajouté} - \text{retiré} &= 4,16 - 6,65 = -2,48 < 0.
 \end{aligned}$$

Comme $\Delta < 0$, le mouvement raccourcit le tour : on l'applique. La nouvelle longueur est $14,92 - 2,48 = 12,44$, et le tour devient $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow A$ (le segment CB a été retourné en BC). C'est exactement le tour de droite de la figure 4 : le croisement a disparu. On répète tant qu'un $\Delta < 0$ existe.

4.3 Le code Python

```

1 def two_opt(tour, D):
2     """Améliore un tour par 2-opt jusqu'à l'optimum local."""
3     n = len(tour)
4     improved = True
5     while improved: # tant qu'on trouve une amélioration
6         improved = False
7         for i in range(1, n - 1):
8             for j in range(i + 1, n):
9                 a, b = tour[i - 1], tour[i]
10                c, d = tour[j], tour[(j + 1) % n]
11                if d == a: # arêtes adjacentes : on saute
12                    continue
13                delta = (D[a][c] + D[b][d]) - (D[a][b] + D[c][d])
14                if delta < -1e-12: # gain strict
15                    tour[i:j + 1] = tour[i:j + 1][::-1] # inversion du segment
16                    improved = True
17     return tour

```

L'inversion `tour[i:j+1] = tour[i:j+1][::-1]` (ligne 14) est précisément le « parcours à l'envers du segment » de la figure 5.

4.4 Preuve que l'algorithme se termine (à savoir par cœur)

Intuition

À chaque mouvement accepté, la longueur du tour *diminue strictement*. Or il n'existe qu'un nombre *fini* de tours possibles. On ne peut donc pas diminuer indéfiniment : forcément, on finit par ne plus trouver d'amélioration. L'algorithme s'arrête.

Argument rigoureux. Soit L_k la longueur du tour après le k -ième mouvement accepté. La suite (L_k) est *strictement décroissante* (on n'accepte que si $\Delta < 0$) et prend ses valeurs dans l'ensemble *fini* des longueurs des $(n-1)!/2$ cycles hamiltoniens. Une suite strictement décroissante dans un ensemble fini est finie : il y a donc un nombre fini de mouvements, et l'algorithme termine. ■

4.5 Complexité, et une honnêteté importante

Une *passé* complète examine toutes les paires (i, j) , soit $O(n^2)$ mouvements candidats, chacun évalué en $O(1)$. Donc $O(n^2)$ par *passé*.

Question piège du jury

« Combien de passes ? Quelle est la complexité *totale* ? » Piège classique. La réponse honnête : on observe *empiriquement* de l'ordre de $O(n)$ passes sur des instances euclidiennes, d'où un total *observé* en $O(n^3)$. Mais ce n'est **pas** un théorème : on peut construire des instances où le nombre de mouvements est exponentiel (Englert, Röglin, Vöcking, 2007 — à vérifier). **Ne réponds jamais** « $O(n^3)$ » **comme si c'était prouvé** ; dis « $O(n^3)$ observé en pratique, sans borne polynomiale démontrée dans le pire cas ».

4.6 Optimum local $\neq$ optimum global (le point crucial)

Définition — optimum local 2-opt

Un tour est un **optimum local** pour 2-opt si *aucun* mouvement à deux arêtes ne l'améliore. Ce n'est pas forcément l'**optimum global** (le meilleur tour absolu) : il peut exister un tour plus court, mais inatteignable en échangeant seulement deux arêtes.

À retenir

Preuve concrète sur mon graphe fil rouge. L'optimum global vaut 11,75 (Held–Karp). Mais si on lance 2-opt depuis le tour de départ [E, A, C, D, F, B], il se bloque sur un tour de longueur 12,07 : aucun échange de deux arêtes ne l'améliore, *pourtant* ce n'est pas l'optimum. C'est l'illustration parfaite de la limite de l'amélioration locale — et un excellent argument à montrer au jury (il prouve que je comprends la différence local/global au lieu de la réciter).

Question piège du jury

« *First-improvement* ou *best-improvement* ? » Deux stratégies : appliquer le *premier* mouvement améliorant trouvé (*first*), ou parcourir tous les candidats et prendre le *meilleur* (*best*). En pratique *first-improvement* converge plus vite pour une qualité comparable. Je dois *tester les deux* sur mes instances et trancher avec des chiffres, pas avec une opinion.

5 Algorithme 3 — Held–Karp (résolution exacte)

5.1 Le gâchis de la force brute

Intuition

La force brute énumère les $(n - 1)!/2$ tours. Le problème n'est pas seulement qu'il y en a beaucoup : c'est qu'elle **recalcule sans cesse les mêmes morceaux**. Le bout de chemin $A \rightarrow B \rightarrow C$ est recalculé pour des *milliers* de tours différents qui le contiennent. Held–Karp supprime ce gâchis en calculant chaque morceau *une seule fois* et en le réutilisant : c'est le principe de la **programmation dynamique**.

5.2 La bonne idée : raisonner sur des sous-ensembles

On fixe un sommet de départ 0. On définit, pour un sous-ensemble S de sommets contenant 0, et un sommet $j \in S$:

Définition — état de Held–Karp

$C(S, j)$ = coût du **plus court chemin** qui part de 0, visite *exactement* les sommets de S , et se termine en j .

Intuition

Au lieu de demander « quel est le meilleur tour complet ? » (trop de réponses), on demande « quel est le meilleur chemin qui visite *ce groupe précis* de points et finit *ici* ? ». Une fois qu'on connaît la réponse pour tous les petits groupes, on construit celle des grands groupes en ajoutant un point à la fois.

5.3 L'équation de récurrence (Bellman)

Pour atteindre j en ayant visité S , on a forcément visité juste avant un autre sommet $i \in S, i \neq j$, en ayant visité $S \setminus \{j\}$. On essaie tous les i possibles et on garde le meilleur :

$$C(S, j) = \min_{\substack{i \in S \setminus \{j\} \\ i \neq 0}} \left[C(S \setminus \{j\}, i) + w(i, j) \right]$$

Cas de base (chemins à deux sommets) : $C(\{0, j\}, j) = w(0, j)$.

Solution finale : une fois la table remplie pour $S = V$ (tous les sommets), on referme le cycle en revenant à 0 :

$$\text{coût optimal} = \min_{j \neq 0} \left[C(V, j) + w(j, 0) \right].$$

Held-Karp : chaque coût $C(S, j)$ est calculé UNE fois puis réutilisé (mémoïsation = principe de la prog. dynamique)

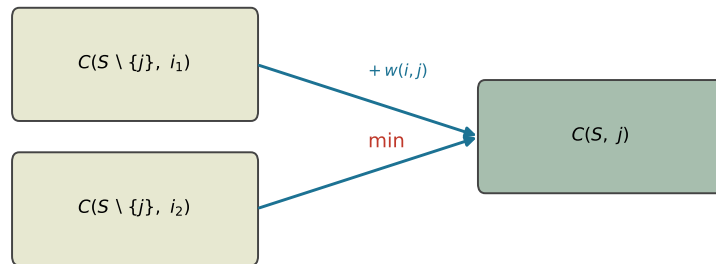


Figure 6: Le cœur de la programmation dynamique : $C(S, j)$ se déduit des $C(S \setminus \{j\}, i)$ déjà calculés, en prenant le minimum. Chaque valeur est calculée une fois, puis mémorisée.

5.4 Comment l'ordinateur représente un sous-ensemble : le « bitmask »

Intuition

Pour stocker $C(S, j)$, l'ordinateur a besoin de représenter le sous-ensemble S par un nombre. Astuce : on écrit un entier en binaire, et le k -ième bit vaut 1 si le sommet k est dans S . Exemple à 4 sommets : $S = \{0, 2, 3\}$ s'écrit 1101 en binaire, soit l'entier 13. Tester « j est-il dans S ? » se fait avec l'opération $S \& (1 \ll j)$. Cela rend le code rapide et compact.

5.5 Déroulé entièrement chiffré sur 4 points

Prenons les quatre premiers points A, B, C, D (départ imposé : A). Les six distances utiles, lues dans la matrice, sont :

$$w(AB) = 2,24, \quad w(AC) = 4,00, \quad w(AD) = 3,45, \quad w(BC) = 2,08, \quad w(BD) = 2,65, \quad w(CD) = 1,92.$$

Étape 1 — cas de base (chemins directs $A \rightarrow j$, donc $C(\{A, j\}, j) = w(A, j)$) :

$$C(\{A, B\}, B) = 2,24, \quad C(\{A, C\}, C) = 4,00, \quad C(\{A, D\}, D) = 3,45.$$

Étape 2 — sous-ensembles de 3 sommets. Comme le départ A est fixé, chaque chemin n'a qu'un seul prédécesseur possible, donc chaque case est une simple addition :

$$\begin{aligned} C(\{A, B, C\}, C) &= C(\{A, B\}, B) + w(BC) = 2,24 + 2,08 = 4,32, & C(\{A, B, D\}, D) &= 2,24 + 2,65 = 4,88, \\ C(\{A, C, D\}, C) &= C(\{A, D\}, D) + w(DC) = 3,45 + 1,92 = 5,37, & C(\{A, C, D\}, D) &= 4,00 + 1,92 = 5,92, \\ C(\{A, B, C\}, B) &= 4,00 + 2,08 = 6,08, & C(\{A, B, D\}, B) &= 3,45 + 2,65 = 6,10. \end{aligned}$$

Étape 3 — ensemble complet, on referme le cycle en ajoutant le retour vers A. Pour chaque sommet final j et chaque prédécesseur i , on calcule $C(\{A, B, C, D\} \setminus \{j\}, i) + w(i, j) + w(j, A)$ et on garde le minimum. Les deux meilleures options (à égalité) :

$$\begin{aligned} \text{fin B, via C : } & \underbrace{5,37}_{C(\{A,C,D\},C)} + \underbrace{2,08}_{w(CB)} + \underbrace{2,24}_{w(BA)} = \mathbf{9,69}, \\ \text{fin D, via C : } & \underbrace{4,32}_{C(\{A,B,C\},C)} + \underbrace{1,92}_{w(CD)} + \underbrace{3,45}_{w(DA)} = \mathbf{9,69}. \end{aligned}$$

(Les autres combinaisons donnent 10,80 ou 12,17 : plus long.) L'optimum vaut donc 9,69, atteint par le cycle $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ (et son symétrique). *Vérification* : la force brute sur les $3! = 6$ tours donne bien 9,69 — Held–Karp a trouvé l'optimum exact, mais en réutilisant des calculs au lieu de tout énumérer.

À retenir

Ce qu'il faut avoir vu défiler ici : (1) on calcule d'abord les *petits* chemins, (2) on construit les *grands* à partir des petits par addition, (3) on referme le cycle à la fin. Aucune permutation n'est énumérée en entier : c'est tout l'intérêt.

5.6 Le code Python

```

1 def held_karp(D):
2     """Resolution EXACTE du TSP par programmation dynamique. Cout  $O(n^2 * 2^n)$ ."""
3     n = len(D)
4     C = {} # C[S, j] = (cout, predecesseur)
5     for j in range(1, n): # cas de base : chemins 0 -> j
6         C[(1 << j, j)] = (D[0][j], 0)
7     for size in range(2, n): # on agrandit S sommet par sommet
8         for subset in subsets(range(1, n), size):
9             bits = sum(1 << k for k in subset) # le sous-ensemble en binaire
10            for j in subset:
11                prev = bits & ~(1 << j) # S prive de j
12                C[(bits, j)] = min(
13                    (C[(prev, i)][0] + D[i][j], i)
14                    for i in subset if i != j) # equation de Bellman
15    full = (1 << n) - 2 # tous les sommets sauf 0
16    cost, parent = min((C[(full, j)][0] + D[j][0], j) for j in range(1, n))
17    # reconstruction du tour en remontant les predecesseurs
18    tour, bits, j = [0], full, parent
19    for _ in range(n - 1):
20        tour.append(j)
21        bits, j = bits & ~(1 << j), C[(bits, j)][1]
22    return tour[:-1], cost

```

5.7 Complexité : pourquoi le mur à $n \approx 20$

Intuition

Il y a un état $C(S, j)$ pour chaque sous-ensemble S (il y en a 2^n) et chaque sommet final j (il y en a n), soit $\sim n 2^n$ états. Calculer chacun demande un minimum sur $\sim n$ prédécesseurs. D'où $n 2^n \times n = n^2 2^n$.

À retenir

Temps : $O(n^2 2^n)$. **Mémoire** : $O(n 2^n)$ (il faut stocker toute la table). C'est souvent la *mémoire* qui sature en premier : pour $n = 20$, $20 \cdot 2^{20} \approx 2 \times 10^7$ entrées (gérable) ; pour $n = 25$, $25 \cdot 2^{25} \approx 8 \times 10^8$ entrées, soit plusieurs Go (rédhibitoire). **Held–Karp est donc inutilisable sur mes 30–40 points** : je l'emploie comme **étalon** sur des sous-graphes de 12 à 15 sommets, pour mesurer à quel point PPV et 2-opt s'écartent de l'optimum *vrai*.

Question piège du jury

« Si Held–Karp donne l'optimum, pourquoi garder les heuristiques ? » Parce qu'il est inapplicable au-delà de $n \approx 20$, alors que mon vrai problème a 30 à 40 points. Sur l'instance réelle, je n'ai *aucun* moyen de calculer l'optimum exact : j'ai besoin d'heuristiques pour produire une solution, et de Held–Karp *uniquement sur des instances réduites* pour estimer leur qualité, puis j'extrapole avec prudence.

6 Synthèse et lien avec le thème « Cycles, boucles »

6.1 Le tableau comparatif (à connaître par cœur)

	Plus proche voisin	2-opt	Held–Karp
Nature	glouton (construit)	amélioration locale	exact (prog. dynamique)
Garantie	aucune	optimum <i>local</i>	optimum <i>global</i>
Temps	$O(n^2)$	$O(n^2)$ / passe	$O(n^2 2^n)$
Mémoire	$O(n)$	$O(n)$	$O(n 2^n)$
Limite pratique	très grand n	très grand n	$n \approx 20$
Rôle dans le TIPE	solution initiale	raffinement	étalon de référence

À retenir

Le compromis fondamental. Pour un problème NP-difficile, on ne peut pas avoir à la fois l'optimalité garantie ET un temps polynomial (sauf si $P = NP$). Mes trois algorithmes occupent trois points de ce compromis : PPV sacrifie la qualité pour la vitesse ; 2-opt sacrifie l'optimalité globale pour rester rapide avec une bonne qualité ; Held–Karp sacrifie le passage à l'échelle pour garantir l'optimum. *C'est pour ça que les comparer a un sens.*

6.2 Le triple ancrage au thème

Le jury testera explicitement le lien avec « Cycles, boucles ». Trois niveaux :

1. **cycle mathématique** : la solution cherchée *est* un cycle hamiltonien ;
2. **boucle algorithmique** : 2-opt est une *boucle* d'amélioration itérative qui tourne tant qu'elle progresse ;
3. **cycle opérationnel** : la collecte est *périodique* (quotidienne) — un cycle temporel.

7 Glossaire express

Sommet (vertex)

un point du graphe ; ici, un point de collecte.

Arête (edge)

un lien entre deux sommets ; ici, un trajet.

Poids (weight)

le coût d'une arête ; ici, un temps de trajet.

Degré

nombre d'arêtes touchant un sommet.

Graphe complet K_n

graphe où toutes les paires de sommets sont reliées.

Cycle eulérien

passé une fois par chaque *arête* (critère d'Euler : degrés pairs).

Cycle hamiltonien

passé une fois par chaque *sommet* ; objet de notre optimisation.

Glouton (greedy)

qui choisit le meilleur coup immédiat sans anticiper.

Heuristique

méthode rapide sans garantie d'optimalité.

Optimum local / global

meilleur dans un voisinage / meilleur en absolu.

Programmation dynamique

résoudre en combinant des sous-problèmes mémorisés.

NP-complet / NP-difficile

classes des problèmes (probablement) sans algorithme polynomial.

Complexité $O(\cdot)$

ordre de grandeur du nombre d'opérations en fonction de n .

TSP / VRP / CARP

voyageur de commerce / tournées de véhicules (capacité) / tournées sur arêtes.

8 Antisèche pour l'oral : 8 questions, 8 réponses

1. **Différence force brute / Held–Karp ?** Les deux sont exacts. Force brute : énumère les $(n-1)!/2$ tours, $O(n!)$. Held–Karp : programmation dynamique sur sous-ensembles, $O(n^2 2^n)$, en factorisant les calculs répétés.
2. **Pourquoi $(n-1)!/2$? $n!$ ordres, divisé par n (point de départ arbitraire sur un cycle) et par 2 (deux sens de parcours).**
3. **2-opt termine-t-il ? Démontrez.** Suite des longueurs strictement décroissante, à valeurs dans l'ensemble fini des $(n-1)!/2$ cycles $\Rightarrow$ nombre fini de mouvements.
4. **2-opt donne-t-il l'optimum ?** Non, un optimum *local*. Contre-exemple chiffré sur mon graphe : bloqué à 12,07 alors que l'optimum est 11,75.
5. **Pourquoi un TSP et pas un CARP ?** Voir §1.6 : simplification de modélisation explicite et assumée, justifiée par la densité des points et le programme MP.
6. **Complexité totale de 2-opt ?** $O(n^2)$ par passe ; nombre de passes $O(n)$ *observé*, total $O(n^3)$ *empirique* — pas de borne polynomiale prouvée.

7. **Held–Karp pour $n = 40$?** $40^2 \cdot 2^{40} \approx 1,8 \times 10^{15}$ opérations et surtout $\sim 4 \times 10^{13}$ cases mémoire : impossible. D'où son rôle d'étalon sur petites instances seulement.
8. **Vos données de Meknès sont-elles fiables ?** Coordonnées relevées par GPS + OpenStreetMap ; temps de trajet *estimés* (vitesse moyenne par type de voie). Ce sont des estimations à valider sur le terrain, pas des mesures exactes — je l'assume comme une limite.

Document de travail — les affirmations marquées « à vérifier » doivent être confirmées dans leur source originale avant la soutenance.